Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de la Información
Boca del Monte, Villa Canales
Programación I
Sección “B”

RESPORTE DE AVANCE NO.2

PROYECTO 01 – TOTITO (3 EN RAYA) MATRIZ 10X10

ESTUDIANTE	**CARNÉ**
Bryan Alejandro Osoy Monterroso	7690-25-15671

INTRODUCCION

Este segundo reporte describe el avance final del Proyecto 01 del curso de Programación 1. Desde el primer reporte se completaron las clases Partida y Main, se integraron todas las clases del sistema y se realizaron pruebas completas del juego. El proyecto se encuentra terminado y funcional al 100%.

En este documento se detallan las funcionalidades implementadas en esta segunda etapa, los problemas encontrados durante la integración de las clases y las soluciones aplicadas.

AVANCE DE CODIFICACION

- **PARTIDA.JAVA: 100%**
  Completada en esta etapa. Guarda y carga partidas en archivo de texto.

- **MAIN.JAVA: 100%**
  Completada en esta etapa. Menú principal, turnos, validaciones y flujo completo.

CLASE PARTIDA: La clase Partida fue la segunda más difícil de implementar. Su función es persistir el estado completo del juego en un archivo de texto llamado 'partida_guardada.txt' para que pueda ser restaurado en cualquier momento.

El método guardarPartida() recibe como parámetros los dos objetos Jugador, la matriz del tablero y el número de turno actual. Escribe cada dato en una línea separada del archivo usando FileWriter y BufferedWriter. Las 10 filas del tablero se escriben como cadenas de 10 caracteres cada una.

El método cargarPartida() hace el proceso inverso: lee el archivo línea por línea con FileReader y BufferedReader y retorna un arreglo de 17 Strings que contienen todos los datos necesarios para reconstruir el juego.

También se implementaron borrarPartida() para eliminar el archivo cuando una partida termina, y existePartida() para verificar si hay una partida guardada antes de intentar cargarla.

CLASE MAIN: La clase Main es la más extensa del proyecto. Contiene el método main() que inicia el programa y varios métodos que manejan el flujo completo del juego.

El menú principal presenta 4 opciones al usuario: nuevo juego, restaurar partida, ver puntajes y salir. Se utilizó un bucle while con un switch para mantener el menú activo hasta que el usuario elija salir.

El método iniciarJuego() contiene el ciclo principal de cada partida. En cada iteración muestra el tablero actualizado, indica de quién es el turno, solicita la jugada mediante fila y columna, valida la entrada y verifica si hay ganador o empate. Si el jugador elige guardar, llama a Partida para persistir el estado.

Se implementó también el método pedirSimbolo() que valida que el símbolo elegido no sea el punto '.' (reservado para casillas vacías) y que no sea igual al símbolo del otro jugador.

PRUEBAS

Una vez completadas las 4 clases se realizaron pruebas completas del sistema. Se probaron los siguientes escenarios:

- Partida normal de inicio a fin con victoria horizontal
- Partida normal de inicio a fin con victoria vertical
- Partida normal de inicio a fin con victoria diagonal
- Empate al llenar el tablero completamente
- Guardar partida a mitad del juego y restaurarla correctamente
- Ingresar coordenadas inválidas (letras, fuera de rango, casilla ocupada)
- Usar símbolos personalizados distintos a X y O
- Jugar múltiples rondas consecutivas y verificar acumulación de puntaje

DIFUCULTADES ENCONTRADAS

La integración de la clase Partida con Main presentó una dificultad interesante: al restaurar el puntaje de los jugadores, no existía un método setPuntaje() en la clase Jugador. Para no modificar el código o arquitectura ya probada, se resolvió llamando al método sumarPunto() en un bucle tantas veces como indicara el puntaje guardado.

Otro problema fue el manejo de excepciones al leer entradas del teclado. Si el usuario ingresaba una letra donde se esperaba un número, el programa lanzaba NumberFormatException y se cerraba. Se resolvió envolviendo todas las lecturas numéricas en bloques try-catch que muestran un mensaje y repiten la solicitud.

También se tuvo que ajustar el método mostrarTablero() para recibir el nombre y carnet del estudiante como parámetros y mostrarlos debajo del tablero en cada turno, cumpliendo con el requisito.

CAPTURAS DE PANTALLA

PARTIDA:

```java
// Clase para guardar y restaurar partidas en archivo de texto
// Autor: [Bryan Alejandro Osoy Monterroso] - Carnet: [7690 25 15671]

import java.io.*;

public class Partida {

    private String nombreArchivo = "partida_guardada.txt";

    // guarda la partida en un archivo de texto
    public void guardarPartida(Jugador j1, Jugador j2, char[][] matriz, int turnoActual) {
        try {
            FileWriter fw = new FileWriter(nombreArchivo);
            BufferedWriter bw = new BufferedWriter(fw);




            // guardamos los datos de los jugadores
            bw.write(j1.getNombre() + "\n");
            bw.write(j1.getSimbolo() + "\n");
            bw.write(j1.getPuntaje() + "\n");
            bw.write(j2.getNombre() + "\n");
            bw.write(j2.getSimbolo() + "\n");
            bw.write(j2.getPuntaje() + "\n");
            bw.write(turnoActual + "\n");

            // guardamos el tablero fila por fila
            for (int i = 0; i < 10; i++) {
                for (int j = 0; j < 10; j++) {
                    bw.write(matriz[i][j] + "");
                }
                bw.write("\n");
            }

            bw.close();
            fw.close();
            System.out.println("Partida guardada correctamente!");

        } catch (IOException e) {
            System.out.println("Error al guardar la partida: " + e.getMessage());
        }
    }

```

```java
// carga la partida del archivo
    // devuelve null si no hay partida guardada
    public String[] cargarPartida() {
        File archivo = new File(nombreArchivo);

        if (!archivo.exists()) {
            System.out.println("No hay ninguna partida guardada.");
            return null;
        }

        try {
            FileReader fr = new FileReader(nombreArchivo);
            BufferedReader br = new BufferedReader(fr);

            // 7 datos de jugadores + turno, mas 10 filas del tablero = 17 lineas
            String[] datos = new String[17];
            for (int i = 0; i < 17; i++) {
                datos[i] = br.readLine();
            }

            br.close();
            fr.close();
            return datos;

        } catch (IOException e) {
            System.out.println("Error al cargar la partida: " + e.getMessage());
            return null;
        }
    }

    // borra el archivo de la partida guardada
    public void borrarPartida() {
        File archivo = new File(nombreArchivo);
        if (archivo.exists()) {
            archivo.delete();
        }
    }

    // revisa si hay una partida guardada
    public boolean existePartida() {
        File archivo = new File(nombreArchivo);
        return archivo.exists();
    }
}

```

MAIN:

```java
// Clase principal
// Proyecto 01 - Totito (3 en raya) en matriz 10x10
// Autor: [BRYAN ALEJANDRO OSOY MONTESORRO] - Carnet: [7690 25 15671]

import java.util.Scanner;

public class Main {

    static Scanner sc = new Scanner(System.in);
    static Tablero tablero = new Tablero();
    static Partida partida = new Partida();
    static Jugador jugador1 = null;
    static Jugador jugador2 = null;
    // 1 para jugador1, 2 para jugador2
    static int turnoActual = 1;

    static String nombreEstudiante = "[Bryan Alejandro Osoy Montesorro]";
    static String carnetEstudiante = "[7690 25 15671]";

    public static void main(String[] args) {
        System.out.println("=== BIENVENIDO AL TOTITO (3 EN RAYA) ===");
        System.out.println("Tablero de 10x10");

        menuPrincipal();
    }

    // menu principal del programa
    public static void menuPrincipal() {
        int opcion = 0;

        while (opcion != 4) {
            System.out.println("\n--- MENU PRINCIPAL ---");
            System.out.println("1. Nuevo juego");
            System.out.println("2. Restaurar partida guardada");
            System.out.println("3. Ver puntajes");
            System.out.println("4. Salir");
            System.out.print("Elige una opcion: ");

            try {
                opcion = Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("Por favor ingresa un numero valido.");
                continue;
            }

            switch (opcion) {
                case 1:
                    configurarNuevoJuego();
                    break;
                case 2:
                    restaurarPartida();
                    break;
                case 3:
                    mostrarPuntajes();
                    break;
                case 4:
                    System.out.println("Gracias por jugar. Hasta luego!");
                    break;
                default:
                    System.out.println("Opcion no valida, intenta de nuevo.");
            }
        }
    }

```

```java
// configura los datos para empezar un juego nuevo
    public static void configurarNuevoJuego() {
        System.out.println("\n--- NUEVO JUEGO ---");

        // pedir nombre del jugador 1
        System.out.print("Nombre del Jugador 1: ");
        String nombre1 = sc.nextLine();

        // pedir simbolo del jugador 1
        char simbolo1 = pedirSimbolo(nombre1, ' ');

        // pedir nombre del jugador 2
        System.out.print("Nombre del Jugador 2: ");
        String nombre2 = sc.nextLine();

        // el jugador 2 no puede usar el mismo simbolo
        char simbolo2 = pedirSimbolo(nombre2, simbolo1);

        // si ya habia jugadores, preguntamos si reiniciar puntajes
        if (jugador1 != null && jugador2 != null) {
            System.out.print("Deseas reiniciar los puntajes? (s/n): ");
            String resp = sc.nextLine();
            if (resp.equalsIgnoreCase("s")) {
                jugador1 = new Jugador(nombre1, simbolo1);
                jugador2 = new Jugador(nombre2, simbolo2);
            } else {
                // actualizamos nombre y simbolo pero mantenemos puntaje
                jugador1.setNombre(nombre1);
                jugador1.setSimbolo(simbolo1);
                jugador2.setNombre(nombre2);
                jugador2.setSimbolo(simbolo2);
            }
        } else {
            jugador1 = new Jugador(nombre1, simbolo1);
            jugador2 = new Jugador(nombre2, simbolo2);
        }

        tablero.limpiarTablero();
        turnoActual = 1;
        iniciarJuego();
    }

    // pide el simbolo al jugador, evita que use el mismo que el otro
    public static char pedirSimbolo(String nombre, char simboloOtro) {
        char simbolo = ' ';
        boolean valido = false;

        while (!valido) {
            System.out.print("Simbolo para " + nombre + " (X, O u otro caracter): ");
            String entrada = sc.nextLine();

            if (entrada.length() == 0) {
                System.out.println("Debes ingresar al menos un caracter.");
                continue;
            }

            simbolo = entrada.charAt(0);

            // verificamos que no sea punto porque lo usamos para casillas vacias
            if (simbolo == '.') {
                System.out.println("El punto (.) esta reservado para casillas vacias. Elige otro.");
                continue;
            }

            // verificamos que no sea igual al otro jugador
            if (simbolo == simboloOtro) {
                System.out.println("Ese simbolo ya lo usa el otro jugador. Elige uno diferente.");
                continue;
            }

            valido = true;
        }

        return simbolo;
    }
```

```java
// aqui se juega la partida
    public static void iniciarJuego() {
        boolean juegoTerminado = false;

        while (!juegoTerminado) {
            Jugador jugadorActual;
            if (turnoActual == 1) {
                jugadorActual = jugador1;
            } else {
                jugadorActual = jugador2;
            }

            tablero.mostrarTablero(nombreEstudiante, carnetEstudiante);
            System.out.println("\nTurno de: " + jugadorActual.getNombre() + " [" + jugadorActual.getSimbolo() + "]");

            // menu durante el juego
            System.out.println("1. Hacer jugada");
            System.out.println("2. Guardar partida");
            System.out.println("3. Salir al menu principal");
            System.out.print("Opcion: ");

            int opcion;
            try {
                opcion = Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("Opcion no valida.");
                continue;
            }

            if (opcion == 2) {
                partida.guardarPartida(jugador1, jugador2, tablero.getMatriz(), turnoActual);
                continue;
            } else if (opcion == 3) {
                System.out.print("Deseas guardar antes de salir? (s/n): ");
                String resp = sc.nextLine();
                if (resp.equalsIgnoreCase("s")) {
                    partida.guardarPartida(jugador1, jugador2, tablero.getMatriz(), turnoActual);
                }
                return;
            } else if (opcion != 1) {
                System.out.println("Opcion no valida.");
                continue;
            }

            // pedir la jugada
            int fila = -1;
            int col = -1;

            try {
                System.out.print("Fila (1-10): ");
                fila = Integer.parseInt(sc.nextLine()) - 1;
                System.out.print("Columna (1-10): ");
                col = Integer.parseInt(sc.nextLine()) - 1;
            } catch (NumberFormatException e) {
                System.out.println("Ingresa numeros validos.");
                continue;
            }

            boolean jugadaOk = tablero.colocarSimbolo(fila, col, jugadorActual.getSimbolo());

            if (!jugadaOk) {
                System.out.println("Casilla ocupada o fuera de rango, intenta de nuevo.");
                continue;
            }

```

```java
// revisar si gano
            if (tablero.hayGanador(jugadorActual.getSimbolo())) {
                tablero.mostrarTablero(nombreEstudiante, carnetEstudiante);
                System.out.println("\n*** " + jugadorActual.getNombre() + " ha ganado! Felicidades! ***");
                jugadorActual.sumarPunto();
                partida.borrarPartida();
                juegoTerminado = true;
                preguntarOtraPartida();
            } else if (tablero.tableroLleno()) {
                tablero.mostrarTablero(nombreEstudiante, carnetEstudiante);
                System.out.println("\nEmpate! El tablero esta lleno.");
                partida.borrarPartida();
                juegoTerminado = true;
                preguntarOtraPartida();
            } else {
                // cambiar turno
                if (turnoActual == 1) {
                    turnoActual = 2;
                } else {
                    turnoActual = 1;
                }
            }
        }
    }

    // pregunta si quieren jugar otra ronda
    public static void preguntarOtraPartida() {
        System.out.print("\nDesean jugar otra partida con los mismos jugadores? (s/n): ");
        String resp = sc.nextLine();
        if (resp.equalsIgnoreCase("s")) {
            tablero.limpiarTablero();
            turnoActual = 1;
            iniciarJuego();
        }
    }

    // restaura una partida guardada
    public static void restaurarPartida() {
        if (!partida.existePartida()) {
            System.out.println("No hay ninguna partida guardada.");
            return;
        }

        String[] datos = partida.cargarPartida();
        if (datos == null) return;

        // reconstruimos los jugadores con los datos guardados
        jugador1 = new Jugador(datos[0], datos[1].charAt(0));
        // no tenemos setter de puntaje, lo sumamos manualmente
        int pts1 = Integer.parseInt(datos[2]);
        for (int i = 0; i < pts1; i++) jugador1.sumarPunto();

        jugador2 = new Jugador(datos[3], datos[4].charAt(0));
        int pts2 = Integer.parseInt(datos[5]);
        for (int i = 0; i < pts2; i++) jugador2.sumarPunto();

        turnoActual = Integer.parseInt(datos[6]);

        // reconstruimos la matriz
        char[][] matrizCargada = new char[10][10];
        for (int i = 0; i < 10; i++) {
            for (int j = 0; j < 10; j++) {
                matrizCargada[i][j] = datos[7 + i].charAt(j);
            }
        }
        tablero.setMatriz(matrizCargada);

        System.out.println("Partida restaurada! Continua jugando...");
        iniciarJuego();
    }

    // muestra los puntajes actuales
    public static void mostrarPuntajes() {
        if (jugador1 == null) {
            System.out.println("Todavia no se ha jugado ninguna partida.");
            return;
        }
        System.out.println("\n--- PUNTAJES ---");
        System.out.println(jugador1.toString());
        System.out.println(jugador2.toString());
    }
}
```

```
PS C:\Users\bryan\OneDrive\UNIVERSIDAD\CICLO 3\PROGRAMACION 1\PROYECTO NO.1>
rs\bryan\AppData\Roaming\Code\User\workspaceStorage\ca8c870d79ed8328e6969090
=== BIENVENIDO AL TOTITO (3 EN RAYA) ===
Tablero de 10x10

--- MENU PRINCIPAL ---
1. Nuevo juego
2. Restaurar partida guardada
3. Ver puntajes
4. Salir
Elige una opcion:
```

```
   1 2 3 4 5 6 7 8 9 10
1  . . . . . . . . . .
2  . . . . . . . . . .
3  . . . . . . . . . .
4  . . . . . . . . . .
5  . . . . . . . . . .
6  . . . . . . . . . .
7  . . . . . . . . . .
8  . . . . . . . . . .
9  . . . . . . . . . .
10 . . . . . . . . . .

[Bryan Alejandro Osoy Montesorro] - [7690 25 15671]
```

Capturas de funcionalidades se estarán mostrando en informe final del proyecto.

CONCLUSION

El Proyecto 01 fue completado satisfactoriamente en su totalidad. Se implementaron las cuatro clases requeridas (Jugador, Tablero, Partida y Main) aplicando correctamente el requisito de Programación Orientada a Objetos.

El proyecto cumple con todas las funcionalidades descritas en el enunciado: tablero de 10×10, dos jugadores con símbolos personalizados, detección de victoria en todas las direcciones, guardado y restauración de partidas.